Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 03

Obtained Marks: _____

Object Oriented Programming

Project Report

Last Date of Submission: 13-4-26

Students Name	Reg No.
Muhammad Anas Abbasi	25108251
Muhammad Haris	25108253
Muhammad Asadullah Saquib	25108252

Submitted To : Hasaan Mujtaba

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Bank Management System

1. Introduction

The Bank Management System is a simple software program that helps manage basic banking operations. It allows users to create bank accounts, deposit money, withdraw money, and check account details easily.

This project is made using Object Oriented Programming (OOP) concepts to show how real-world systems like banks can be designed using programming.

2. Project Idea (Concept)

The main idea of this project is to create a system that can handle multiple bank accounts in an organized way.

The system helps users to:

- Create new bank accounts
- Deposit money
- Withdraw money
- View account details

It also ensures that all operations are safe and properly managed.

3. Problem Statement

In real life, manual banking systems can cause many problems such as: •

Slow processing of data

- Human errors in calculations
- Difficulty in managing multiple accounts

So, we need a digital system that is:

- Fast

- Accurate
- Easy to use

4. Features of the System

1. Create Account

User can create a new account by entering: •

Account number

- Name
- Initial balance

2. Deposit Money

- User can add money to their account
- Balance updates automatically

3. Withdraw Money

- User can withdraw money
- System checks balance before withdrawing •

Prevents overdrawing

4. Display Account Details

Shows:

- Account number
- Account holder name
- Current balance

5. Show Total Accounts

- Displays total number of accounts created •

Uses static variable

6. Multiple Account Handling

- Stores multiple accounts using array of objects •

Helps manage data efficiently

5. OOP Concepts Used

1. Encapsulation

- Data is kept private
- Access through functions like deposit(), withdraw() •

Protects data from direct access

2. Abstraction

- User interacts with simple functions
- Hides complex implementation

3. Array of Objects

- Used to store multiple accounts
- Makes management easier

4. Static Variable

- Shared among all objects
- Tracks total accounts

5. Function Overloading

- Same function name with different parameters •

Improves flexibility

6. Usefulness of the System

This system is useful because:

- It manages banking activities easily
- Allows quick transactions
- Stores all account data in one place
- Reduces manual work
- Provides fast access to information

7. Benefits of the System

1. Saves Time

- Fast transactions
- No manual calculations

2. Accuracy

- Reduces human errors
- Automatic balance updates

3. Easy Record Management

- Stores multiple accounts
- Data is easy to access

4. Data Security

- Uses encapsulation
- Prevents unauthorized access

5. Scalability

- Can be expanded with more features
- Example: money transfer, loan system

6. Learning Benefits

- Helps understand real-world OOP use
- Improves programming skills

8. Conclusion

The Bank Management System is a simple and effective program that helps manage banking operations efficiently. It saves time, reduces errors, and keeps data secure.

It is also very helpful for students to understand how OOP concepts are used in real-life applications.

```
#include <iostream>
using namespace std;

class BankAccount {
private:
int accNumber;
string name;
double balance;

static int totalAccounts;

public:
BankAccount() {
accNumber = 0;
name = "";
balance = 0;
}

void createAccount(int num, string n, double bal) {
accNumber = num;
name = n;
balance = bal;
totalAccounts++;
}

void deposit(double amount) {
balance += amount;
cout << "Amount deposited successfully\n";
}

void deposit(int amount) {
balance += amount;
cout << "Amount deposited successfully\n";
}

void withdraw(double amount) {
if (amount <= balance) {
balance -= amount;
```

```
cout << "Withdrawal successful\n";
} else {
cout << "Insufficient balance\n";
}
}

void display() {
cout << "\nAccount Number: " << accNumber << endl;
cout << "Name: " << name << endl;
cout << "Balance: " << balance << endl;
}

int getAccNumber() {
return accNumber;
}

static void showTotalAccounts() {
cout << "Total Accounts: " << totalAccounts << endl;
}
};

int BankAccount::totalAccounts = 0;

int main() {
BankAccount accounts[5];
int count = 0;
int choice;

do {
cout << "\n--- Bank Management System ---\n";
cout << "1. Create Account\n";
cout << "2. Deposit\n";
cout << "3. Withdraw\n";
cout << "4. Display All Accounts\n";
cout << "5. Show Total Accounts\n";
cout << "0. Exit\n";
cout << "Enter choice: ";
cin >> choice;
```

```
int accNum;
string name;
double amount;

switch (choice) {
case 1:
if (count < 5) {
cout << "Enter Account Number: ";
cin >> accNum;
cout << "Enter Name: ";
cin >> name;
cout << "Enter Initial Balance: ";
cin >> amount;

accounts[count].createAccount(accNum, name, amount);
count++;
} else {
cout << "Bank storage full!\n";
}
break;

case 2:
cout << "Enter Account Number: ";
cin >> accNum;
cout << "Enter Amount: ";
cin >> amount;

for (int i = 0; i < count; i++) {
if (accounts[i].getAccNumber() == accNum) {
accounts[i].deposit(amount);
break;
}
}
break;

case 3:
cout << "Enter Account Number: ";
```

```
cin >> accNum;
cout << "Enter Amount: ";
cin >> amount;

for (int i = 0; i < count; i++) {
if (accounts[i].getAccNumber() == accNum) {
accounts[i].withdraw(amount);
break;
}
}
break;

case 4:
for (int i = 0; i < count; i++) {
accounts[i].display();
}
break;

case 5:
BankAccount::showTotalAccounts();
break;

}
} while (choice != 0);

return 0;
}
```

Default output

```
--- Bank Management System ---
1. Create Account
2. Deposit
3. Withdraw
4. Display All Accounts
5. Show Total Accounts
0. Exit
```

Create Account

```
Enter choice: 1
Enter Account Number: 109
Enter Name: Anas
Enter Initial Balance: 10000
Account Created Successfully
```

Deposit Amount

```
Enter choice: 2
Enter Account Number: 109
Enter Amount: 2000
Amount deposited successfully
```

Withdraw Amount

```
Enter choice: 3
Enter Account Number: 109
Enter Amount: 1000
Withdrawal successful
```

Display Account

```
Enter choice: 4

Account Number: 109
Name: Anas
Balance: 11000
```

Total Accounts

```
Enter choice: 5
Total Accounts: 1
```

Exit

```
Enter choice: 0
Exiting Program...
```